

GISELE — Proposta Backend v1.0

GISELE — Proposta de Arquitetura Backend

Versão: 1.0 (Maio 2026) **Contexto:** análise da plataforma GISELE v2.6.0 (~13.400 linhas em HTML+JS single-page) com identificação de candidatos para migração ao backend, proposta de arquitetura distribuída e cronograma de migração faseada. **Autor:** análise técnica para CPTEC/INPE

Sumário Executivo

O GISELE hoje é um aplicativo full-client (single-page HTML + Electron) que executa **100 % do processamento no navegador**: decodificação de GeoTIFF, cálculos de raster algebra (algébrica + temporal), recorte de área, exportação de GeoJSON, fetch + cache de TIFs do FTP do CPTEC. Esse modelo funcionou bem na fase de prototipagem mas atingiu limites claros à medida que as features cresceram:

- 1 **Performance:** decodificar um TIF de 5 MB do BESM (1440×721 Float32) leva ~600 ms; a Calculadora Temporal `sum(t1..t24)` precisa fazer isso 24 vezes (~15 s só de decode), tudo no main thread. Operações como exportar GeoJSON sobre campo global geram arquivos de centenas de MB e travam o browser.
- 2 **Compartilhamento:** cada usuário re-baixa e re-decodifica os mesmos TIFs do CPTEC. Numa equipe de 20 meteorologistas analisando a mesma rodada do Eta, isso são 20× fetch + 20× decode + 20× paleta. Há economia óbvia em cache compartilhado.
- 3 **Persistência:** todo o estado do usuário (modelos configurados, anotações, séries temporais extraídas) vive em `localStorage` por máquina. Trocar de computador perde tudo. Análises feitas por um pesquisador não são reaproveitadas pela equipe.
- 4 **Integração:** GISELE é uma ilha — não há como Grafana, Jupyter Notebook ou um modelo downstream consumir os mesmos dados (TIFs decodificados, séries temporais extraídas, GeoJSONs exportados) sem refazer todo o pipeline.

Esta proposta apresenta uma migração **incremental** para uma arquitetura de **microsserviços políglotas** (Python para ciência, Go para tile/cache, Node para gateway), com **dois modos de deploy** suportados:

- **Modo Central** — servidor compartilhado do CPTEC; ideal para equipes internas que precisam colaboração e cache compartilhado;
- **Modo Local** — `docker compose up` ao lado do Electron; mantém autossuficiência do app mas tira o trabalho pesado do main thread do browser.

O frontend HTML+Electron **não é descartado** — ele continua sendo o cliente. A migração é incremental: serviço a serviço, com fallback para o código in-browser durante toda a transição. Em ~18 semanas é possível ter os 7 serviços principais operacionais com o cliente migrado para chamadas REST.

1. Análise da arquitetura atual

1.1. Estrutura monolítica

O GISELE consiste em:

- `figuras_SisMOM_v23.html` (1,2 MB, 13.369 linhas) — single-page HTML com JavaScript embarcado em IIFEs no `<script>` final.
- `electron-app/main.js` (29 linhas) — main process do Electron; apenas cria a janela e expõe flags de CLI.
- `miscelaneas/*.geojson` — camadas vetoriais embarcadas inline via `<script type="application/json">`.

Todo o processamento é client-side. Não há servidor — o app fala direto com o FTP do CPTEC via `fetch()`.

1.2. Estado persistido

O estado do usuário vive em `localStorage` do navegador:

Chave	Conteúdo
<code>sismom_state_png</code>	snapshot do estado da aba PNG/GIF (modelo, data, passo, layout)
<code>sismom_state_gtiff</code>	snapshot da aba GeoTIFF
<code>sismom_models_v2</code>	configuração de modelos e variáveis (CRUD do usuário)
<code>sismom_gt_sections</code>	preferências de UI (sidebar aberta/fechada, etc)
<code>gt.navHud</code>	toggle do HUD lat/lon/valor
Configurações de paletas, min/max default, anotações	vários

Não há sincronização entre máquinas. Limite prático: ~5 MB por origem.

1.3. Funções pesadas (candidatas naturais a backend)

A análise do código identificou as seguintes funções como CPU- ou IO-intensivas:

Função	Linha	Custo dominante	Frequência
<code>SisMOM_GeoTIFF.decodeTIFF(buffer)</code>	6076	decodificação de IFD + descompressão LZW + flatten para Float32Array (CPU bound, ~600 ms p/ 1440×721)	A cada novo passo da animação, troca de modelo, troca de data
<code>aplicarPaleta(decoded, opts)</code>	6338	iteração pixel-a-pixel aplicando lookup de paleta + máscara NoData + clipping (~150 ms p/ 1 Mpx)	A cada mudança de paleta/min-max/clip
<code>_gtFetchAndDecode(url)</code>	6441	fetch HTTP + decode TIF + cache em RAM (LRU)	Idem decodeTIFF
<code>gtSampleProfile(slotIdx, coords)</code>	7135	sampling adaptativo ao longo de polilinha; reuso de decoded em RAM	Por interação do usuário
<code>gtSampleTimeSeries(slotIdx, lat, lon)</code>	7440	loop sequencial: para cada passo da rodada (até 720h ÷ freq) faz fetch + decode + amostra ponto (lat,lon). É a função mais cara de toda a aplicação.	Por clique do usuário
<code>gtCreateLayerFromExpression(expr, ...)</code>	11239	parse de expressão + eval per-pixel sobre N camadas (CPU bound)	Por clique em Calcular
<code>gtCreateLayerFromTimeExpression(expr, ...)</code>	11429	identifica passos requisitados, fetch + decode de cada, eval per-pixel	Por clique em Aplicar na linha Tempos
<code>_gtParseShpBuffer(arrayBuffer)</code>	11678	parser puro JS do shapefile (110 linhas)	Por upload

Função	Linha	Custo dominante	Frequência
<code>_gtExtractFromZip(buffer, ext)</code>	11783	leitor de ZIP + DecompressionStream	Por upload
<code>gtExportLayerToGeoJsonPointCloud(...)</code>	11986	varre todos os pixels do raster, point-in-polygon, monta features GeoJSON. Pode produzir arquivos de 100+ MB.	Por clique em Exportar
<code>gtSampleTimeSeriesToGeoJson(...)</code>	12078	idem <code>gtSampleTimeSeries</code> + serialização	Por clique em Exportar série temporal
<code>gravarVideoEvolucaoTemporal()</code>	4210	pre-fetch de todos os frames + draw em canvas off-screen + MediaRecorder	Por clique em Salvar MP4
Marching squares para contornos	—	algoritmo O(N) sobre o grid; UI em real-time	A cada troca de paleta/threshold

1.4. Limitações observadas

- **Re-trabalho:** vários usuários decodificando o mesmo TIF em paralelo. Cada um paga ~600 ms + memória de buffer Float32.
- **Memória do main thread:** abrir 3 painéis com Eta-3km (5000×3000) carrega ~60 MB de Float32 só de raster, mais paletas + ImageBitmaps + cache. Aba GISELE consome facilmente 1-2 GB de RAM.
- **CORS:** o FTP do CPTEC não envia headers CORS, então o Electron precisa rodar com `webSecurity: false` (vetor de risco moderado) para gravar MP4.
- **Estado isolado:** as configurações de modelos que um meteorologista refina (templates de URL, paletas default por variável, etc.) ficam presas no `localStorage` da máquina dele.

2. Categorias de candidatos a backend

Os candidatos podem ser agrupados em **7 categorias**, cada uma com objetivos e métricas próprias:

#	Categoria	Funções atuais	Benefício principal
1	Decodificação de raster	<code>decodeTIFF</code> , <code>aplicarPaleta</code>	Performance, cache compartilhado
2	Cache de TIFs / tile cache	<code>_gtFetchAndDecode</code> , prefetch	Compartilhamento entre usuários
3	Calculadora (raster algebra)	<code>gtCreateLayerFromExpression</code> , <code>gtCreateLayerFromTimeExpression</code>	Performance + reproducibilidade
4	Sampling / extração de dados	<code>gtSampleTimeSeries</code> , <code>gtSampleProfile</code> , exportar GeoJSON, série temporal	Performance + exportação assíncrona
5	Conversão de formatos / GIS	<code>_gtParseShpBuffer</code> , <code>_gtExtractFromZip</code> , contornos vetorizados	Robustez (libs maduras), suporte a mais formatos
6	Persistência de estado	<code>localStorage</code> de modelos, anotações	Sincronização entre máquinas, colaboração

#	Categoria	Funções atuais	Benefício principal
7	API pública / integração	(não existe hoje)	Reaproveitamento por outras ferramentas do CPTEC

Nas próximas seções cada categoria é detalhada como um **serviço** independente.

3. Arquitetura proposta

3.1. Visão geral



3.2. Princípios de design

- 1 **Frontend continua sendo o cliente.** O HTML+Electron não é descartado — vira um cliente "fino" que delega o trabalho pesado mas mantém toda a interatividade local (pan, zoom, anotações).
- 2 **Microserviços políglotas.** Cada serviço usa a stack mais apropriada para seu domínio (Python p/ ciência, Go p/ I/O e cache, Node p/ orquestração e estado).
- 3 **Stateless quando possível.** Os serviços de Decoder, Calc, Export e Format não guardam estado de sessão — qualquer instância pode atender qualquer requisição. Isso simplifica scaling horizontal e Docker compose local.
- 4 **Cache multi-camada.** Redis para o que é quente (últimos 100 TIFs decodificados), MinIO para o que é morno (TIFs decodificados retidos por dias), e o FTP do CPTEC como ground truth.
- 5 **API REST primeiro, SSE para progresso.** REST cobre 95 % dos casos. Operações de longa duração (calc temporal sobre 24 passos, exportar GeoJSON global) usam Server-Sent Events para empurrar progresso.

- 6 **Modo Local idêntico ao Modo Central.** Os mesmos containers Docker rodam no laptop do usuário (`docker compose up`) ou no servidor do CPTEC. A configuração muda só o `compose.yml`.
- 7 **Fallback in-browser preservado.** Durante toda a migração, o cliente verifica se o backend está disponível; se não, faz fallback para o código JS atual. Permite rollout gradual sem interrupção.

4. Detalhamento por serviço

4.1. Serviço de Decodificação de Raster (`raster -decoder`)

Stack: Python 3.11 + FastAPI + rasterio + numpy + pillow.

Responsabilidade: decodificar GeoTIFFs e aplicar paletas no servidor, devolvendo arrays binários ou imagens PNG prontas.

Endpoints:

```
POST /v1/decoder/decode
body: { url: "https://ftp1.cptec.inpe.br/.../prec-024.tif" }
→ 200 { id: "sha256:...", width, height, bbox: {minX,minY,maxX,maxY},
      nodata, dtype, min, max, percentiles: [p1, p5, p50, p95, p99],
      dataUrl: "/v1/decoder/data/sha256:..." }

GET /v1/decoder/data/{id}
→ 200 (application/octet-stream) – Float32Array raw bytes

POST /v1/decoder/palette
body: { id: "sha256:...", paleta: "viridis", min, max,
      nodataExtras: [-9999, 1e20], clipBelow, clipAbove }
→ 200 (image/png) – bitmap pronto para overlay
```

Pontos críticos:

- **Cache por URL** — antes de decodificar, computa SHA-256 da URL e checa Redis. Se hit, devolve o id existente. TTL de 24h em Redis + persistência longa em MinIO.
- **Heurísticas legacy preservadas** — multi-sentinel NoData (range > 1e6), auto flip-Y por tiepoint J, `GTRasterTypeGeoKey` `PixelsPoint/Area` precisam ser portadas do JS para Python. `rasterio` lida com a maioria automaticamente, mas detalhes da heurística do GISELE precisam de testes de regressão.
- **Paletas idênticas ao frontend** — as 15 paletas atuais (Viridis, Jet, RdBu, ...) viram um pacote Python compartilhado para que o output seja **byte-idêntico** ao do JS. Senão, mudar de in-browser para backend altera as cores percebidas pelo meteorologista.
- **Resposta em pedaços** — para um TIF gigante, devolver 60 MB de Float32 em uma resposta REST é ruim. Alternativa: o `decode` retorna só metadados + um endpoint streamável (`/data/{id}`) que devolve por chunks. Ou usar Arrow/Parquet binário.

Benefícios mensuráveis:

- Cache compartilhado: 20 usuários acessando o mesmo TIF pagam 1× decode (~600 ms) + 19× hit de cache (~1 ms).
- Main thread liberado: o browser recebe um PNG pronto + metadados, sem precisar alocar Float32Array de 12 MB para um TIF de 1440×721.

Métrica de aceitação: P95 de decode < 1 s para TIFs do FTP CPTEC; bitmap idêntico ao output JS em ≥ 99.99 % dos pixels (tolerância de arredondamento).

4.2. Serviço de Tile Cache + Proxy FTP (`tile-cache`)

Stack: Go + chi router + groupcache.

Responsabilidade: ser o **único** componente que conversa com o FTP do CPTEC, oferecer cache HTTP padrão com `ETag/Last-Modified`, e resolver o problema de CORS de uma vez por todas.

Endpoints:

```
GET /v1/tiles/cptec/{model}/{yyyy}/{mm}/{dd}/{hh}/{kind}/{filename}
```

→ 200 (image/tiff ou image/png) com headers CORS abertos
→ cabeçalhos: ETag, Last-Modified, Cache-Control: public, max-age=86400

GET /v1/tiles/health

→ 200 { ftp_reachable: true, cache_hit_rate: 0.87, cached_files: 12345 }

Pontos críticos:

- **Proxy single-source-of-truth.** Todo fetch que hoje vai pro FTP passa por aqui. Permite (1) cache compartilhado, (2) headers CORS uniformes (resolve a necessidade de `webSecurity: false` no Electron), (3) telemetria centralizada (quais modelos são mais consumidos), (4) blindar contra mudança de URL no FTP.
- **Cache em duas camadas:** RAM (groupcache, ~10 GB hot) + disco (MinIO ou FS local com LRU policy, ~500 GB warm).
- **Resiliência:** quando o FTP fica fora do ar (acontece), o tile-cache continua servindo a partir do cache. Status `503 Service Unavailable` só quando recém-consultado e ainda não cacheado.
- **Conversão opcional:** o endpoint pode aceitar `?format=cog` e converter para Cloud Optimized GeoTIFF on-the-fly (rasterio do raster-decoder), permitindo range requests do browser sem decode completo.

Benefícios mensuráveis:

- 1 fetch ao FTP por modelo+passo+data (em vez de N por usuário).
- Latência média de TIF cai de ~800 ms (FTP, banda do CPTEC) para ~30 ms (cache local).
- Remove a necessidade da flag `--strict-cors` no Electron.

Métrica de aceitação: cache hit rate > 80 % após 1 semana de operação; tempo de fetch < 100 ms no P95 em cache hit.

4.3. Serviço de Calculadora (calc-engine)

Stack: Python 3.11 + FastAPI + xarray + numpy + numexpr.

Responsabilidade: executar Calculadora Algébrica (entre camadas) e Temporal (entre passos da mesma rodada) no servidor; devolver o resultado como um raster decodificado novo (mesmo formato do `raster-decoder`).

Endpoints:

```
POST /v1/calc/algebraic
  body: { expression: "Camada1 * 1000 + Camada2",
          layers: { Camada1: "sha256:...", Camada2: "sha256:..." } }
→ 202 Accepted { jobId: "calc_abc123" }
```

```
POST /v1/calc/temporal
  body: { expression: "sum(t1..t24)",
          modelId: "Eta3km", variavelId: "prec",
          dataRodada: "2026052800" }
→ 202 Accepted { jobId: "calc_xyz789" }
```

```
GET /v1/calc/{jobId}/stream (Server-Sent Events)
→ event: progress data: {done: 5, total: 24, currentStep: "t5"}
→ event: progress data: {done: 12, total: 24, currentStep: "t12"}
→ event: result data: {id: "sha256:...", min, max, bbox}
→ event: error data: {message: "..."}

```

Pontos críticos:

- **Engine em Python.** O parser recursive-descent atual (em JS) precisa ser reimplementado em Python — ou substituído pelo já maduro `numexpr` que suporta a maioria das operações. Para a sintaxe temporal `tN/hN/range/sum/mean/max/min`, o parser custom continua sendo mais simples (~200 linhas Python).
- **Vetorização real.** O JS atual avalia pixel-a-pixel em laço `for`. Em Python com numpy, é uma operação vetorial → 10-50× mais rápido. `sum(t1..t24)` vira `np.sum(stack, axis=0)` direto.
- **Cancelamento.** O SSE permite o cliente abortar (mesmo behavior do botão Cancelar atual). O job é interrompido entre passos, libera memória, finaliza.

- **Persistência opcional.** O resultado da calc fica disponível por X horas como camada (id = sha256). Útil para colaboração: um meteorologista calcula `sum(t1..t72)` e compartilha o id com o time.

Benefícios mensuráveis:

- Calc temporal `sum(t1..t72)` no Eta-3km: hoje ~45 s no browser (24× decode + 24× iteração JS). No backend, com cache de decode + numpy vetorial: ~5 s.
- Browser não trava — operação roda assíncrona com progresso via SSE.

Métrica de aceitação: speedup $\geq 5\times$ sobre o equivalente in-browser para expressões com range ≥ 12 passos; resultado byte-idêntico ao da implementação atual (tolerância $\epsilon = 1e-6$).

4.4. Serviço de Exportação (export-service)

Stack: Python 3.11 + FastAPI + GeoPandas + fiona + shapely.

Responsabilidade: exportar dados em formatos diversos. Hoje o cliente só faz GeoJSON; o backend amplia para vários formatos.

Endpoints:

```
POST /v1/export/raster
  body: { decodedId: "sha256:...", format: "geojson" | "geoparquet" |
    "csv" | "netcdf" | "shapefile",
    mask: { type: "polygon", coords: [[lat,lon],...] } |
    { type: "rect", bbox: [...] } |
    { type: "layerId", id: "geojson_xyz" } |
    null }
  → 202 { jobId, statusUrl }
```

```
POST /v1/export/timeseries
  body: { modelId, variavelId, dataRodada, lat, lon,
    format: "geojson" | "csv" | "json" }
  → 202 { jobId, statusUrl }
```

```
GET /v1/export/{jobId}
  → 200 { status: "running"|"done"|"error", progress: 0.45,
    resultUrl: "/v1/export/{jobId}/download" (quando done) }
```

```
GET /v1/export/{jobId}/download
  → 200 application/* - arquivo final pronto
```

Pontos críticos:

- **Formatos extras.** Além de GeoJSON (já implementado client-side), o backend pode oferecer GeoParquet (compacto, ideal para análise downstream em Python/Spark), NetCDF (formato padrão da meteorologia), Shapefile, CSV. Isso fecha o ciclo com Jupyter Notebooks da equipe.
- **Streaming.** Para campos globais, o GeoJSON resultado pode ser 100 MB+. O endpoint `/download` faz streaming (chunked) para evitar carregar tudo em RAM no servidor.
- **Lifecycle.** Exports ficam guardados por 24h e depois são GC-ados. Se o usuário quiser preservar, integra com o serviço de Persistência (`user-state`).
- **GeoPandas robust handling.** Substituir o parser de shapefile in-browser por `geopandas.read_file()` no backend dá robustez sobre encoding (CP1252 vs UTF-8 no .dbf), projeções (.prj com EPSG variados, reprojeção para EPSG:4326), e MultiPolygon edge cases que o parser puro JS simplifica.

Benefícios mensuráveis:

- Exportar campo cheio do Eta-3km: hoje ~15-20 s + arquivo de 80 MB que pode travar download no browser. No backend: roda assíncrono, browser recebe URL para download progressivo.
- Novos formatos (NetCDF, GeoParquet) abrem a porta para integração com Jupyter/Spark sem reimplementação.

Métrica de aceitação: P95 de export de campo global < 30 s; arquivo final $\leq 110\%$ do tamanho do equivalente in-browser; teste de round-trip (export $\rightarrow$ re-import) preserva valores no $\epsilon = 1e-6$.

4.5. Serviço de Conversão de Formatos ([format-converter](#))

Stack: Python 3.11 + FastAPI + GDAL + rasterio + fiona.

Responsabilidade: centralizar conversões entre formatos GIS, complementando [export-service](#). Útil quando o usuário sobe um arquivo num formato não suportado pelo browser.

Endpoints:

```
POST /v1/convert/upload
  multipart/form-data: file=<arquivo>
  → 200 { uploadId, detectedType: "shapefile_zip"|"geojson"|"kml"|"gpkg"|...,
        features: 1234, bbox: [...], crs: "EPSG:4326" }

POST /v1/convert/reproject
  body: { uploadId, targetCRS: "EPSG:4326" }
  → 200 { newId }

GET /v1/convert/{id}/as/geojson
  → 200 application/geo+json
```

Pontos críticos:

- **Suporte amplo.** GDAL suporta 100+ formatos. Isso permite ao usuário subir KML (Google Earth), GPKG, Shapefile com qualquer encoding/CRS, e o backend devolve GeoJSON normalizado em EPSG:4326.
- **Reprojeção.** Hoje o GISELE assume EPSG:4326. Se um usuário sobe um shape em UTM SAD-69, o resultado fica deslocado. O [/reproject](#) corrige isso transparentemente.
- **Validação.** Shapefiles com geometrias inválidas (auto-interseção, anéis abertos) são corrigidos com [shapely.make_valid\(\)](#) antes de serem servidos.

Benefícios mensuráveis:

- Remove ~170 linhas de parser shapefile JS atual; troca por uma chamada REST.
- Aceita formatos antes inviáveis (KML do Google Earth é comum entre pesquisadores).
- Resolve bug de CRS que hoje obriga o usuário a converter manualmente no QGIS.

Métrica de aceitação: aceita pelo menos 8 formatos GIS comuns; reprojeção de qualquer EPSG conhecido para 4326 sem perda de geometria; arquivo de teste regression suite de 50 shapes diversos passa 100 %.

4.6. Serviço de Estado / Colaboração ([user-state](#))

Stack: Node.js 22 + Fastify + PostgreSQL + Prisma ORM.

Responsabilidade: persistir o que hoje vive em [localStorage](#) (configurações de modelos, anotações, layouts) e habilitar colaboração entre usuários.

Endpoints:

```
GET    /v1/state/models           → lista de modelos do usuário
POST   /v1/state/models         → cria/atualiza
DELETE /v1/state/models/{id}

GET    /v1/state/annotations?slot=0 → anotações do slot
POST   /v1/state/annotations       → cria
DELETE /v1/state/annotations/{id}

POST   /v1/state/share         → marca um item (modelo/anotação/calc result)
  body: { itemId, scope: "team"|"public" }
  → URL pública: "https://gisele.cptec.inpe.br/share/{token}"

GET    /v1/state/sessions/{userId} → estado completo da última sessão
POST   /v1/state/sessions       → snapshot
```

Schema PostgreSQL (simplificado):

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email TEXT UNIQUE, name TEXT, role TEXT,
  created_at TIMESTAMPTZ DEFAULT now())
```



```

);

CREATE TABLE model_configs (
  id UUID PRIMARY KEY,
  owner_id UUID REFERENCES users(id),
  name TEXT, json JSONB,
  shared_with_team BOOLEAN DEFAULT false,
  updated_at TIMESTAMPTZ
);

CREATE TABLE annotations (
  id UUID PRIMARY KEY,
  owner_id UUID REFERENCES users(id),
  slot_idx INT, payload JSONB,
  geom GEOMETRY(Geometry, 4326), -- usa PostGIS
  created_at TIMESTAMPTZ
);

CREATE TABLE saved_calcs (
  id UUID PRIMARY KEY,
  owner_id UUID REFERENCES users(id),
  expression TEXT, description TEXT,
  result_decoded_id TEXT, -- referência ao raster-decoder
  created_at TIMESTAMPTZ
);

```

Pontos críticos:

- **Sincronização.** Cliente Electron tem cliente offline-first: lê do [localStorage](#), sincroniza com o backend quando online. Conflitos resolvidos com last-write-wins por padrão (configurável).
- **Identidade.** Auth via OAuth do CPTEC (já existem credenciais corporativas); JWT para autorização. Para Modo Local, usuário único default sem login.
- **Compartilhamento.** Um meteorologista pode publicar "Configuração Eta SUDESTE" e o time inteiro consome — sem cada um refazer.
- **Migração suave.** O cliente faz upload one-shot do [localStorage](#) atual no primeiro login, populando o backend.

Benefícios mensuráveis:

- Troca de máquina = login = todo o estado de volta.
- Configurações de modelo refinadas por especialistas viram patrimônio do time.
- Anotações em mapas viram base de conhecimento institucional.

Métrica de aceitação: P95 de leitura de estado < 200 ms; sync transparente sem conflitos perceptíveis ao usuário em workflow normal.

4.7. API Gateway ([gateway](#))

Stack: Node.js 22 + Fastify + JWT + OpenTelemetry.

Responsabilidade: roteamento, autenticação, rate-limit, observabilidade. Ponto único de entrada do cliente.

Funcionalidades:

- **Roteamento por path:** [/v1/decoder/*](#) → raster-decoder; [/v1/tiles/*](#) → tile-cache; etc.
- **Auth:** valida JWT do CPTEC OAuth; injeta [X-User-Id](#) para os serviços downstream.
- **Rate-limit:** 100 req/min por usuário em endpoints de calc/export (operações caras).
- **CORS:** uniforme para todos os serviços.
- **Telemetria:** traces OpenTelemetry encadeados; métricas Prometheus (latency, error rate, request count por endpoint).
- **Health check agregado:** [/health](#) retorna status de cada serviço downstream.

Métrica de aceitação: overhead < 5 ms em request normal; traceability 100 % de qualquer request pelo trace ID.

5. Modos de deploy

5.1. Modo Central (servidor CPTEC)

```
# docker-compose.prod.yml
services:
  gateway:      { image: cptec/gisele-gateway:2.6 }
  raster-decoder: { image: cptec/gisele-decoder:2.6, replicas: 4 }
  tile-cache:    { image: cptec/gisele-tile:2.6, replicas: 2 }
  calc-engine:   { image: cptec/gisele-calc:2.6, replicas: 4 }
  export-service: { image: cptec/gisele-export:2.6, replicas: 2 }
  format-converter: { image: cptec/gisele-format:2.6 }
  user-state:    { image: cptec/gisele-state:2.6 }
  postgres:      { image: postgres/postgis:16 }
  redis:         { image: redis:7-alpine }
  minio:         { image: minio/minio }
```

- **Backend:** servidor único do CPTEC; 16 vCPU / 64 GB RAM / 2 TB SSD comporta ~100 usuários simultâneos.
- **Frontend:** os usuários acessam o GISELE como **PWA web** (mesmo HTML, sem Electron) apontando para <https://gisele.cptec.inpe.br>.
- **Auth obrigatória** via OAuth corporativo.

5.2. Modo Local (laptop do usuário)

```
# docker-compose.local.yml – mesmo set, sem replicas
services:
  gateway:      { image: cptec/gisele-gateway:2.6, ports: ["8080:8080"] }
  raster-decoder: { image: cptec/gisele-decoder:2.6 }
  tile-cache:    { image: cptec/gisele-tile:2.6, volumes: ["/cache:/cache"] }
  calc-engine:   { image: cptec/gisele-calc:2.6 }
  export-service: { image: cptec/gisele-export:2.6 }
  format-converter: { image: cptec/gisele-format:2.6 }
  user-state:    { image: cptec/gisele-state:2.6 }
  postgres:      { image: postgres/postgis:16, volumes: ["/pgdata:/var/lib/postgresql/data"] }
  redis:         { image: redis:7-alpine }
```

- **Backend:** roda ao lado do Electron via `docker compose up`. Auto-start via systemd/Windows Service.
- **Frontend:** continua sendo o Electron. Aponta para <http://localhost:8080>.
- **Auth desabilitada** (single-user default).
- **Tile cache** persiste em disco local — o usuário acumula seu próprio cache offline ao longo do tempo.

A grande vantagem desse design: **o cliente é o mesmo HTML** em ambos os modos. Só muda a variável `GISELE_BACKEND_URL`. A maior parte do código que migrou para o backend continua funcionando se o backend estiver indisponível (fallback in-browser).

6. Plano de migração faseado

Migração proposta em **6 fases** ao longo de ~18 semanas. Cada fase entrega valor independentemente — se a migração parar em qualquer ponto, o GISELE continua funcionando.

Fase 0 — Preparação (Semanas 1-2)

- Setup do monorepo (`gisele-backend/`) com pastas por serviço.
- CI/CD: GitHub Actions buildando imagens Docker, push para registry interno.
- Standup de ambiente de dev em `docker compose` na máquina dos devs.
- Especificação OpenAPI 3.1 dos endpoints de todos os serviços (escrita antes de qualquer código).

- Definição da política de versionamento de API (`/v1/`, `/v2/`).

Entregável: repo + CI + specs OpenAPI. Sem código de serviço ainda.

Fase 1 — Tile Cache (Semanas 3-5)

O caso mais simples e de maior impacto imediato. **Não muda nada no cliente** — só vira proxy.

- Implementar `tile-cache` em Go.
- Deploy em modo Central como proxy do FTP.
- Cliente passa a apontar para `tile-cache` em vez de FTP direto.
- **Quick win:** resolve CORS, melhora latência média, remove dependência da flag `--strict-cors`.

Entregável: clientes (Electron + futuro web) usando o tile-cache em produção.

Fase 2 — Raster Decoder + Estado (Semanas 6-9)

- `raster-decoder` em Python. Porta de `decodeTIFF` + `aplicarPaleta`. Testes de regressão garantindo paridade byte-a-byte com o JS.
- `user-state` em Node. Schema PostgreSQL. Migração one-shot do `localStorage`.
- Auth via OAuth CPTec (opcional nesta fase; default sem auth).
- Cliente: usa `decoder` para overlays; fallback in-browser se backend indisponível.

Entregável: decoder rodando + estado persistido + cliente híbrido.

Fase 3 — Calculadora (Semanas 10-12)

- `calc-engine` em Python. Suporte completo a Algébrica + Temporal + funções de redução.
- SSE para progresso.
- Cliente: substitui `gtCreateLayerFromTimeExpression` por chamada REST com progresso visual.

Entregável: Calculadora Temporal `sum(t1..t72)` em 5 s em vez de 45 s.

Fase 4 — Export + Format Converter (Semanas 13-15)

- `export-service` em Python. GeoJSON + GeoParquet + NetCDF + Shapefile + CSV.
- `format-converter` em Python. Aceita KML, GPKG, Shapefile, GeoJSON; converte para GeoJSON normalizado.
- Cliente: substitui parsers in-browser por chamadas REST. Remove ~200 linhas de código JS.

Entregável: export multiformato + import robusto. Integração com Jupyter dos cientistas.

Fase 5 — Gateway + Observabilidade + Hardening (Semanas 16-17)

- `gateway` Node.js — todos os requests passam por aqui.
- OpenTelemetry instrumentation em todos os serviços.
- Dashboard Grafana com métricas chave (latency P95, error rate, cache hit rate).
- Auth JWT obrigatória em endpoints sensíveis.
- Rate-limit por usuário.

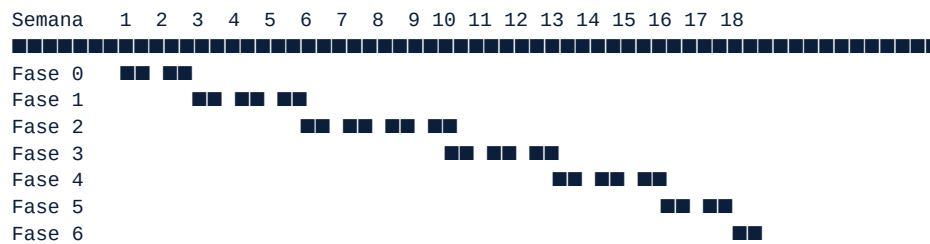
Entregável: observabilidade de ponta a ponta + hardening pré-produção.

Fase 6 — Modo Local + Documentação (Semana 18)

- `docker-compose.local.yml` testado e documentado.
- Installer para Windows/Mac/Linux que sobe Docker compose ao lado do Electron.
- Documentação operacional (deploy, monitoramento, troubleshooting).
- HANDOVER atualizado.

Entregável: Modo Local production-ready + docs.

Cronograma visual



7. Análise de riscos e mitigações

Risco	Probabilidade	Impacto	Mitigação
Mudança de output (paleta/decoder não byte-idêntico ao JS) confunde usuários acostumados	Alta	Médio	Testes de regressão automatizados comparando outputs JS vs Python; release notes detalhadas; toggle para forçar legacy in-browser durante 1 release
FTP do CPTEC fica fora do ar e tile-cache é o único caminho	Média	Alto	Tile-cache serve do cache local; alerta para SRE; SLO de retenção mínima de 7 dias para rodadas recentes
Migração de localStorage para PostgreSQL causa perda de configurações	Baixa	Alto	Migração lazy + dry-run no primeiro login; backup do localStorage em arquivo <code>.json</code> antes de qualquer escrita ao backend
Performance do calc-engine Python pior do que esperado para grids enormes (e.g. global 0.25°)	Média	Médio	Benchmark cedo (semana 10) com Eta-3km + ICON global; se ruim, fallback para Numba/Cython ou Go para o hot path
Auth do CPTEC complica deploy em Modo Local	Média	Baixo	Modo Local pula auth (single-user default); só Modo Central exige login
Dependência de GDAL no format-converter cria container pesado (~500 MB)	Alta	Baixo	Multi-stage build; usar osgeo/gdal:alpine-small como base; tradeoff aceitável dado a robustez
Equipe não tem expertise Go (para tile-cache)	Média	Médio	Tile-cache em Python como alternativa (perda de ~30 % de performance, mas redução de complexidade)

Risco	Probabilidade	Impacto	Mitigação
Cliente híbrido (REST + fallback in-browser) duplica código durante a transição	Alta	Médio	Feature flag global; quando o backend de uma feature estabilizar, remove o fallback JS daquela feature; manter apenas durante a transição

8. Métricas de sucesso

8.1. Performance

- **Decode time P95:** hoje ~600 ms in-browser; meta ≤ 50 ms (cache hit) ou ≤ 1 s (cache miss).
- **Calc temporal** `sum(t1..t72)`: hoje ~45 s; meta ≤ 8 s.
- **Export global GeoJSON:** hoje ~20 s + browser pode travar; meta ≤ 30 s assíncrono com browser responsivo.
- **Tempo de carregar primeira tela do GISELE:** hoje ~3 s (HTML 1.2 MB + parsing); meta ≤ 1.5 s.

8.2. Compartilhamento

- Cache hit rate do tile-cache: meta > 80 % após 2 semanas em produção.
- Redução de banda do FTP do CPTEC consumida pelo GISELE: meta ≥ 70 % (dado 20 usuários acessando rodadas comuns).
- Configurações de modelo compartilhadas pelo time: meta ≥ 30 % das configurações em `team` scope.

8.3. Adoção

- % de instalações Electron usando backend (vs full client): meta ≥ 90 % em 8 semanas após deploy de Modo Local.
- # de chamadas à API externa (Jupyter, Grafana) por semana: meta ≥ 500 após 12 semanas.

8.4. Operação

- Disponibilidade da API: meta 99.5 % em horário comercial CPTEC; 99 % 24/7.
- MTTR (Mean Time To Recovery) em incidentes: meta < 30 min.
- Cobertura de testes automatizados: meta ≥ 80 % dos serviços Python; ≥ 70 % Go/Node.

9. O que NÃO migrar

Para evitar over-engineering, algumas coisas devem **continuar no cliente**:

- **Rendering do canvas + interações UI** — pan, zoom, anotações em tempo real, draws de polígono/retângulo. Latência baixa precisa estar local.
- **Tile providers (Esri, OSM, OpenTopo)** — esses já são CDNs globais com cache otimizado. Proxy não agregaria valor.
- **HUD de lat/lon/valor** — precisa ser instantâneo no movimento do mouse; impossível fazer round-trip para o backend.
- **Cache curto de overlay** — manter ImageBitmap do passo atual no browser para animação suave.
- **State de UI volátil** — qual nó da árvore está expandido, qual paleta está selecionada no select. Só estado durável vai pro backend.

A regra empírica: **se a operação demora < 16 ms (1 frame a 60 fps), fica no cliente.**

10. Conclusão e próximos passos

A migração para microsserviços políglotas resolve as quatro dores principais do GISELE:

- 1 **Performance** — operações pesadas (decode, calc temporal, export) saem do main thread.
- 2 **Compartilhamento** — cache no servidor evita re-trabalho entre usuários; configurações de modelo viram patrimônio.
- 3 **Persistência** — estado não morre com o navegador; troca de máquina preserva tudo.
- 4 **Integração** — API REST expõe os dados decodificados / calculados / exportados para o resto do ecossistema CPTEC.

A proposta é **incremental e reversível**. Em qualquer fase, é possível parar — o sistema continua funcionando, apenas com menos serviços migrados. Não há ponto de não-retorno arquitetural.

Próximos passos imediatos (próximos 5 dias úteis):

- 1 Validar essa proposta com líderes técnicos do CPTEC (1 dia).
- 2 Decidir sobre OAuth corporativo (compartilhamento de identidade) vs auth standalone (1 dia).
- 3 Levantar capacidade do servidor para deploy Central (1 dia).
- 4 Definir SLOs precisos com a equipe operacional (1 dia).
- 5 Setup do monorepo e CI inicial — Fase 0 começa (1 dia).

Repositório: [C:\Projetos\Visualizador](#) · **GISELE versão analisada:** v2.6.0 · **Documento de referência:** [HANDOVER_GISELE.md](#), [ESPECIFICACOES_GISELE.md](#)